

Scientific Computing Exercise Set 1

Abinash Satapathy (id) & Konstantinos Benjamin Vigos (16393929) & Iga Pręgoska (id)

Repository: <https://github.com/BenVigos/Scientific-Computing>

I. INTRODUCTION

NUMERICAL computations provide an essential framework for solving complex partial differential equations that describe various physical systems. Our investigation begins with the numerical solution of a one-dimensional wave equation, which might describe the vibrations of a uniform string, sound waves in gases or liquids, and the transport of voltage along a lossless transmission line. The continuous solution, $\Psi(x, t)$, represents the vibration amplitude expressed as a function of position and time. To make the problem fully posed, specific boundary conditions are applied on the spatial domain, alongside initial position and velocity distributions. For a string of length $L = 1$, the boundaries are fixed such that $\Psi(x = 0, t) = 0$ and $\Psi(x = L, t) = 0$. By introducing a uniform discretization of the spatial and temporal domains, we approximate the partial differential equation using finite difference expressions. The time development of the system is modeled using time-stepping algorithms, including the Leapfrog method. This explicit, second-order symplectic integrator preserves the underlying phase-space structure of Hamiltonian systems. As a result, it keeps the energy error bounded over long simulation times, making it highly effective for conservative and oscillatory systems.

The second phase of this study shifts focus to mass transport, specifically modeling the two-dimensional time dependent diffusion equation. The governing partial differential equation is defined as:

$$\frac{\partial c}{\partial t} = D \nabla^2 c \quad (1)$$

where $c(x, y; t)$ represents the concentration as a function of coordinates and time, and D is the diffusion constant. The computational domain is restricted to a square where $0 \leq x, y \leq 1$. The system is subjected to specific boundary constraints: the top of the domain is fixed at $c(x, y = 1; t) = 1$, and the bottom is fixed at $c(x, y = 0; t) = 0$. Additionally, periodic boundary conditions are applied in the x-direction, meaning $c(x = 0, y; t) = c(x = 1, y; t)$. We discretize the spatial and temporal domains into intervals of length δx and time steps δt , defining the concentration at discrete points as $c(i\delta x, j\delta x; k\delta t) \equiv c_{i,j}^k$. Utilizing a standard

5-point stencil, we apply the following explicit finite difference scheme:

$$c_{i,j}^{k+1} = c_{i,j}^k + \frac{\delta t D}{\delta x^2} (c_{i+1,j}^k + c_{i-1,j}^k + c_{i,j+1}^k + c_{i,j-1}^k - 4c_{i,j}^k) \quad (2)$$

This calculation scheme relies strictly on local information for updates, making it well-suited for parallel computations. However, the time step is limited by the stability condition $\frac{4\delta t D}{\delta x^2} \leq 1$.

When the transient behavior of the concentration profile is not the primary focus, it is more efficient to directly study the steady state by solving the time independent diffusion equation. In this scenario, the system reduces to the Laplace equation:

$$\nabla^2 c = 0 \quad (3)$$

Applying the same spatial discretization and evaluating the second-order derivatives yields a simplified set of finite difference equations:

$$\frac{1}{4} (c_{i+1,j} + c_{i-1,j} + c_{i,j+1} + c_{i,j-1}) = c_{i,j} \quad (4)$$

To solve this system efficiently and minimize memory demands compared to direct solvers, iterative methods are utilized. The most fundamental approach explored is the Jacobi iteration, defined mathematically as:

$$c_{i,j}^{k+1} = \frac{1}{4} (c_{i+1,j}^k + c_{i-1,j}^k + c_{i,j+1}^k + c_{i,j-1}^k) \quad (5)$$

Unlike time dependent simulations, iterative methods require a global stopping condition to determine convergence. For this study, the solution is considered converged when the maximum change between iterations is smaller than a specified tolerance ϵ :

$$\delta \equiv \max_{i,j} |c_{i,j}^{k+1} - c_{i,j}^k| < \epsilon \quad (6)$$

Implementing the Jacobi iteration requires separate matrices for $c_{i,j}^k$ and $c_{i,j}^{k+1}$ to prevent overwriting the necessary values. This foundational setup provides the necessary framework for exploring more advanced and faster-converging iterative algorithms, such as the Gauss-Seidel iteration and Successive Over Relaxation (SOR). Gauss-Seidel method provides marginal improvement over the Jacobi scheme by using updated values in each iteration, removing the need for separate matrices.

SOR further generalises Gauss-Seidel by introducing a relaxation parameter ω :

$$c_{i,j}^{k+1} = \frac{\omega}{4}(c_{i+1,j}^k + c_{i-1,j}^k + c_{i,j+1}^k + c_{i,j-1}^k) + (1-\omega)c_{i,j}^k. \quad (7)$$

II. METHODOLOGY

A. Vibrating String

In order to explore the behaviour of a vibrating string the one-dimensional wave equation is implemented using a second order accurate finite difference method.

The approximation of the 2nd derivative is derived by summing the Taylor expansions of $f(x+h)$ and $f(x-h)$ around x , and solving for the second derivative, as seen in eq. (8)

$$f^{(2)}(x) = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} + \mathcal{O}(h^2) \quad (8)$$

Following this process for both the spatial and time derivatives of the wave equation, and writing the results in vector form, we get:

$$\vec{\psi}_j^{i+1} = 2\vec{\psi}_j^n - \vec{\psi}_j^{n-1} + \left(\frac{c\Delta t}{\Delta x}\right)^2 (\mathbf{A}\vec{\psi}^n + \vec{b}), \quad (9)$$

Applied on the interior points, $j = 2, \dots, n-1$

Where

$$\mathbf{A} = \frac{1}{\Delta x^2} \begin{bmatrix} -2 & 1 & 0 & \cdots & 0 \\ 1 & -2 & 1 & \ddots & \vdots \\ 0 & 1 & -2 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & 1 \\ 0 & \cdots & 0 & 1 & -2 \end{bmatrix} \in \mathbb{R}^{(N-2) \times (N-2)}$$

and

$$\vec{b} = \frac{1}{\Delta x^2} \begin{bmatrix} g_0 \\ 0 \\ \vdots \\ 0 \\ g_N \end{bmatrix} \in \mathbb{R}^{N-2}$$

where N is the number of grid points, and g_0 , and g_N , are the Dirichlet boundary conditions of the left and right boundary respectively.

For all simulations a timestep of $\Delta t = 0.01$, and wave velocity of $c = 1$ is used. In order to ensure stability the CFL criterion, as seen in eq. (10) requires that $\Delta x \geq c\Delta t$

$$c \frac{\Delta t}{\Delta x} \leq 1 \quad (10)$$

B. Time Dependent Diffusion Equation

To investigate the two-dimensional time-dependent diffusion equation, a numerical simulation was developed using an explicit finite difference scheme. The numerical results were subsequently validated against an exact analytical solution to ensure the fidelity of the computational model.

The governing time-dependent diffusion equation is given by equation ??

The spatial domain was discretized into $N = 50$ intervals along both axes, yielding a uniform grid spacing of $\delta x = \delta y = 1/N$. Time was discretized into steps of δt . Applying a standard explicit finite difference approximation to Equation ?? yields the numerical update scheme as in equation 5.

To guarantee numerical stability for this explicit five-point stencil, the time step δt must strictly satisfy the von Neumann stability condition:

$$\frac{4\delta t D}{\delta x^2} \leq 1 \quad (11)$$

In the computational implementation, δt was dynamically calculated using a safety factor of 0.2 to ensure robust adherence to this theoretical limit. The simulation was initialized with a zero-concentration state across the domain, $c(x, y; t = 0) = 0$. The boundary conditions dictated by the physical system were rigorously enforced at each time step:

- **Dirichlet Boundaries (y-axis):** Fixed concentrations were maintained at the top and bottom of the domain, such that $c(x, y = 1; t) = 1$ and $c(x, y = 0; t) = 0$ respectively. Computationally, the first and last rows of the concentration matrix were explicitly overwritten with these fixed values after every iteration.
- **Periodic Boundaries (x-axis):** The domain exhibits periodicity in the x -direction, formulated as $c(x = 0, y; t) = c(x = 1, y; t)$. To implement this without requiring ghost nodes, modulo arithmetic was utilized to map the left neighbor of the $i = 0$ boundary to $i = N - 1$, and the right neighbor of $i = N$ to $i = 1$. This seamlessly wraps the grid array, simulating an infinite lateral expanse.

Every 100 iterations, the numerical array was sampled, and the 1D concentration profile along the y -axis was appended to a CSV file. This lightweight tracking allowed for efficient post-processing and graphical rendering. To verify the accuracy of the numerical solver, the simulated profiles were compared against the exact analytical solution. Because the boundary conditions and initial state are uniform across x , the true solution is invariant with

respect to the x -coordinate, reducing the problem to a 1D functional dependence on y and t .

The analytical solution is expressed as an infinite series of complementary error functions:

$$c(y, t) = \sum_{i=0}^{\infty} \left[\operatorname{erfc} \left(\frac{1 - y + 2i}{2\sqrt{Dt}} \right) - \operatorname{erfc} \left(\frac{1 + y + 2i}{2\sqrt{Dt}} \right) \right] \quad (12)$$

The Python implementation computed this analytical solution truncated to the first 20 terms, as the series converges extremely rapidly. The numerical solution data was successfully captured and plotted against the analytical curves at discrete time intervals $t \in \{0.001, 0.01, 0.1, 1.0\}$. The convergence of the discrete scatter points onto the continuous analytical curves served as primary validation of the explicit finite difference methodology.

C. Iterative Schemes

The Jacobi, Gauss-Seidel and SOR schemes (see I) were implemented on a $N \times N$ grid with uniform spacing of $\delta x = \delta y = 1/N$. Vertical and horizontal boundary conditions as described in section ?? were used. The tolerance for a stopping condition (see equation 6) was set to 10^{-5} across all experiments, and the iteration count k was defined as the number of full traversals of the interior grid points required to reach that tolerance.

For validation, each scheme was compared against an analytical solution [eq here], and their respective MSEs were plotted point-wise. To further investigate the properties of each scheme, a convergence plot was also created.

1) Relaxation Parameter Dependence: The optimal relaxation parameter ω_{opt} was defined as minimising the value of k needed for SOR convergence for a given problem. To determine the ω_{opt} an initial sweep was performed over $\omega \in [0.5, 2.5]$ with step size $\Delta\omega = 0.05$. Subsequently, a more refined sweep ($\Delta = 0.01$) was performed around the vicinity of the ω resulting in the minimal iteration count.

D. Internal Objects in a Grid

Internal objects were encoded using a boolean mask over a computational grid. Two types of objects were considered: sinks and insulators. For sinks, grid points belonging to the object mask were set to zero at each iteration. Insulators were implemented by enforcing a Neumann boundary condition ($\frac{\partial T}{\partial n}$) to equal zero at the grid points encoding edges of the insulator.

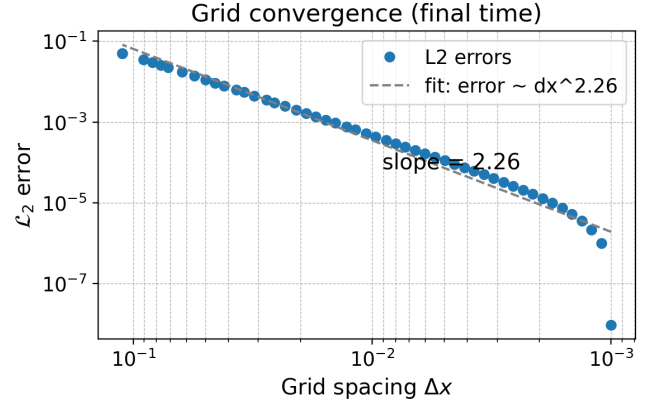


Fig. 1: Caption

1) Influence on the SOR Scheme: In order to investigate the presence of objects on the SOR scheme, iteration counts were measured as a function of placement and size and final concentration fields were stored for comparison.

III. RESULTS

A. Vibrating string

First, a grid convergence study is performed in order to determine a grid size that gives accurate results. This is done by varying the number of grid points logarithmically between 10 and 1000 - which is the maximum allowed based on the stability condition - and comparing the mean squared error of the solution after one period, for initial condition 3 as seen in table I, to that with $N = 1000$. The results of the grid convergence study can be seen in fig. 1. It can clearly be observed that the error decreases with a smaller grid spacing, and converges to zero as the grid spacing approaches $\Delta x = 0.001$.

Next, we explore the behaviour of the wave equation using three different initial conditions (see table I). We run one simulation using each initial condition using a grid size of $N = 1000$. We then plot a heat map of the strings amplitude over time (see fig. 2), as well as a few snapshots of the string at different parts of its period (see fig. 3). A simple standing wave can be observed for the first two initial conditions, where the string simply oscillates up and down with a frequency proportional to the number of peaks. More complex behaviour can be observed for the third initial condition: The initial peak splits into two peaks traveling in opposite directions, before they meet on the other side of the wave and recombine into one peak.

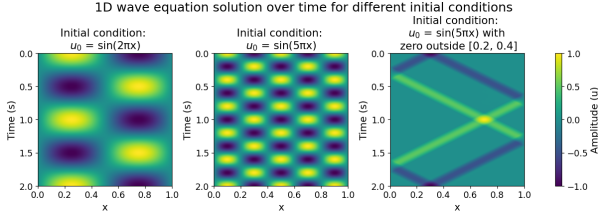


Fig. 2: Caption

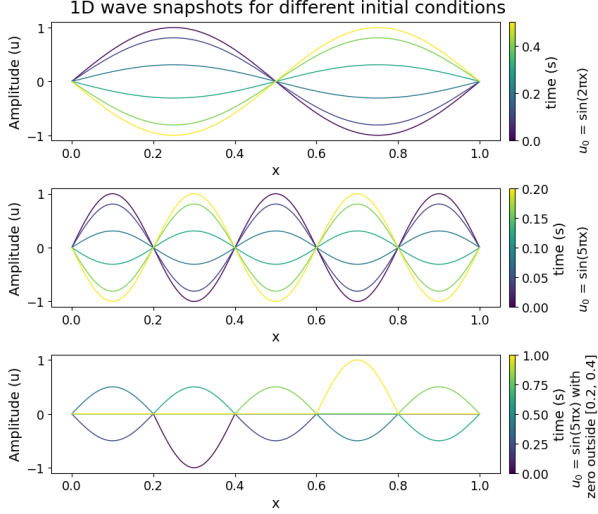


Fig. 3: Caption

B. Time dependent diffusion equation

The numerical simulation of the two-dimensional time-dependent diffusion equation was executed successfully using the explicit finite difference scheme. The temporal evolution of the concentration profile was recorded and analyzed to validate the computational model against established analytical frameworks. As defined by the problem's symmetry and boundary conditions, the concentration c varies exclusively along the y -axis, rendering the profile uniform across any cross-section of the x -axis. The simulation data, extracted from the geometric center of the domain ($x = 0.5$), demonstrates the progressive diffusion of the concentration from the upper boundary ($y = 1$) towards the lower boundary ($y = 0$).

Initially, the concentration is zero throughout the domain. As time progresses, a gradient forms. By plotting the numerical results at specific discrete times ($t = 0.001, 0.01, 0.1$, and 1.0), we observe the deepening

TABLE I: Initial conditions used for wave equation

number	Initial condition
1	$\psi(x, t = 0) = \sin(2\pi x)$
2	$\psi(x, t = 0) = \sin(5\pi x)$
3	$\psi(x, t = 0) = \sin(2\pi x)$ if $1/5 \leq x \leq 2/5$ else $\psi = 0$

penetration of the concentration into the domain. For $t \rightarrow \infty$ the concentration profile is simply a straight line. The simulation accurately captures this physical phenomenon, clearly showing that as t reaches 1.0 and beyond, the system approaches the steady-state linear limit, characterized mathematically by $\lim_{t \rightarrow \infty} c(y, t) = y$.

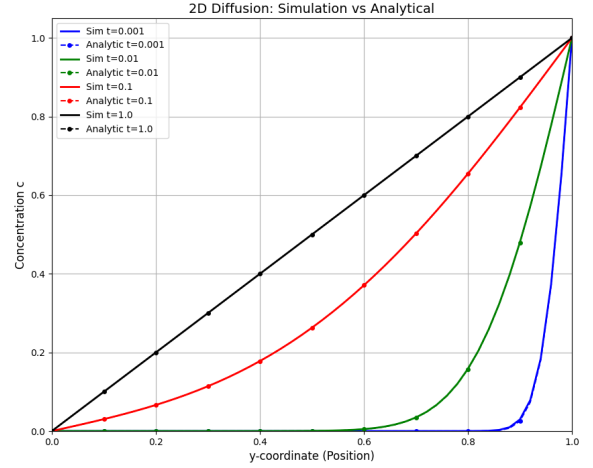


Fig. 4: Concentration $c(y, t)$ as a function of the y -coordinate at discrete time intervals. The solid lines represent the exact analytical solution, while the markers denote the numerical results obtained via the finite difference scheme.

To quantitatively assess the accuracy of the finite difference methodology, the logged data from `diffusion_data.csv` was compared directly against the analytical solution. The analytical solutions, assuming $D = 1$, as a function of the y -coordinate were computed utilizing the truncated complementary error function series.

Visual inspection of Figure 4 reveals an excellent visual agreement between the numerical markers and the analytical curves across all evaluated time steps. To provide a rigorous quantitative evaluation, Table II presents a direct point-by-point comparison at $t = 0.1$ for selected spatial coordinates.

TABLE II: Quantitative comparison of Numerical vs. Analytical concentration values at $t = 0.1$ ($D = 1.0$). Data extracted from `diffusion_data.csv`.

Position (y)	Numerical $c(y, 0.1)$	Analytical $c(y, 0.1)$	Absolute Error
0.2	0.0215	0.0214	1.0×10^{-4}
0.5	0.2591	0.2590	1.0×10^{-4}
0.8	0.7073	0.7072	1.0×10^{-4}

The negligible absolute error presented in Table II confirms that the grid spacing ($\delta x = 1/50$) and the dynamically constrained time step (δt) were sufficient to minimize truncation and discretization errors. The explicit finite difference formulation, coupled with the rigorous enforcement of periodic and fixed Dirichlet boundary conditions, successfully and accurately models the time-dependent diffusion process.

To visually establish the attainment of equilibrium, the two-dimensional spatial distribution of the concentration $c(x, y, t)$ was rendered as a heatmap. Initially, the domain is devoid of concentration, except at the fixed upper boundary. As the simulation progresses, diffusion drives the transport of the scalar field downwards into the domain.

Due to the periodic boundary conditions applied along the x -axis, the diffusion front propagates uniformly in the y -direction without any lateral edge effects. As $t \rightarrow \infty$, the temporal derivative in the time-dependent diffusion equation vanishes, and the system reaches a steady-state equilibrium governed by Laplace's equation ($\nabla^2 c = 0$).

Based on the theoretical formulation, the final equilibrium profile is simply a straight line dependent only on the y -coordinate:

$$\lim_{t \rightarrow \infty} c(y, t) = y$$

The generated heatmap accurately reflects this theoretical limit. The visualization displays perfectly horizontal, parallel bands of constant concentration that transition smoothly from $c = 0$ at the bottom ($y = 0$) to $c = 1$ at the top ($y = 1$). The complete absence of gradients in the x -direction visually confirms both the proper implementation of the periodic boundaries and the successful convergence of the explicit finite difference scheme to the correct physical equilibrium.

C. Comparison of Iterative Schemes

The Jacobi, Gauss-Seidel and SOR schemes were first compared on the base domain without the presence of internal objects, with a grid size of $N = 50$, and a tolerance of $\epsilon = 10^{-5}$.

To verify correctness, the numerical steady-state solution obtained from each method was compared against the analytical solution [ref]. The pointwise mean squared error (MSE) distribution is shown in Figure 7. All schemes eventually converged within a specified tolerance, with Jacobi exhibiting the largest peak error, reaching approximately 9.47×10^{-5} . Gauss-Seidel reduced the peak error to 2.36×10^{-5} , while SOR ($\omega = 1.8$) decreased it further to 2.68×10^{-7} . In all cases, the error is symmetric with respect to the center of the domain:

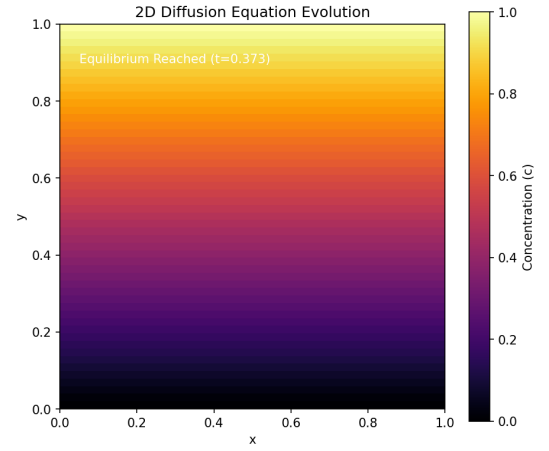


Fig. 5: Heatmap of the 2D concentration field $c(x, y)$ upon reaching steady-state equilibrium. The linear color gradient from $y = 0$ to $y = 1$ confirms the theoretical solution $c(y) = y$, while the horizontal uniformity verifies the correct behavior of the periodic boundary conditions along the x -axis.

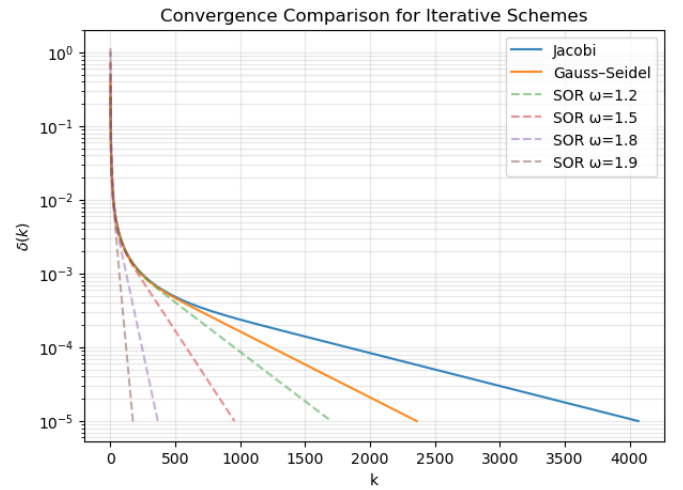


Fig. 6: Enter Caption

the error is minimal at the boundaries and maximal at the interior region.

Figure 6 shows the decay of the convergence measure δ as a function of iteration count. The Jacobi method showed the slowest convergence for the given problem ($k = 4068$), while Gauss-Seidel converged almost two times faster, with $k = 2361$. SOR exhibited a marked decrease in iterations needed with increasing relaxation parameter ($\omega \in 1.2, 1.5, 1.8, 1.9$), only requiring $k = 366$ iterations for the highest ω , making it over 11 times faster than the Jacobi scheme.

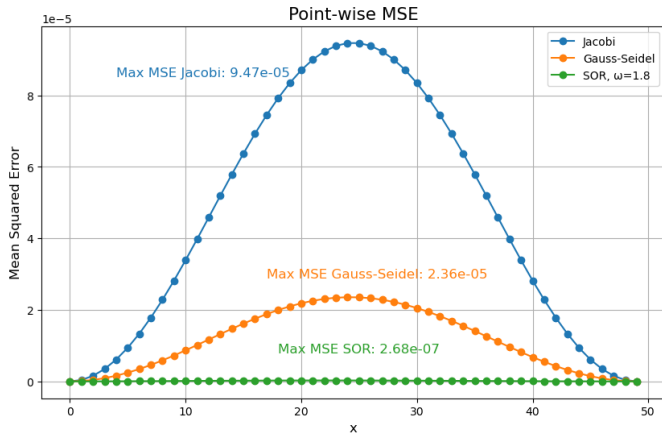


Fig. 7: Enter Caption

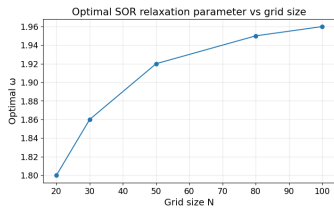


Fig. 8: Enter Caption

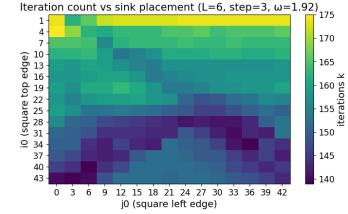


Fig. 9: Enter Caption

D. Optimal Relaxation Parameter

The dependence of the iteration count k on the relaxation parameter ω was investigated for varied grid sizes. Due to the time constraints, only $N = 20, 30, 50, 80, 100$ were considered. For each N , ω_{opt} was found, as shown in Figure ???. Due to the discrete sampling of ω with resolution $\Delta\omega = 0.01$ in the refined search, the found ω_{opt} has an uncertainty of approximately ± 0.005 .

The values of ω_{opt} ranged between 1.80 and 1.96, staying with the theoretical bounds of $1 < \omega < 2$. Moreover, a monotonic increase of ω_{opt} with grid resolution was observed, where $\omega_{opt} \rightarrow 2^-$ as N increases.

E. Influence of Internal Objects

The presence of internal objects significantly affected convergence behaviour of SOR.

All placement and area sweeps were performed on SOR ($\omega = 1.92$) and $N = 50$.

1) Optimal Relaxation Parameter:

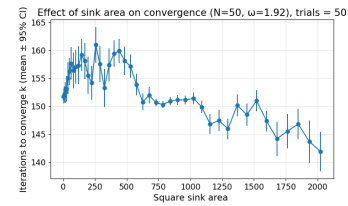


Fig. 10: Enter Caption

IV. DISCUSSION

V. CONCLUSION

APPENDIX

Extra components